Variational Auto Encoders (VAEs)

2020.10.14

Generative models

1. Generative Adversarial Networks(GANs)：
2. Variational Autoencoders(VAEs)：
   - ☐ VAE
   - ☐ beta_VAE
   - ☐ IAF (inverse autoregressive flow)
   - ☐ VAE with GAN
   - ☐ iVAE (identifiable VAE)
3. Autoregressive models
   - ☐ MADE
   - ☐ PixelRNN

VAE

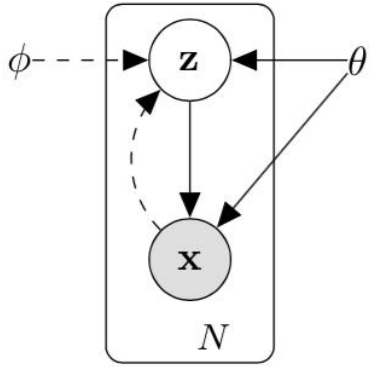
Auto-Encoding Variational Bayes

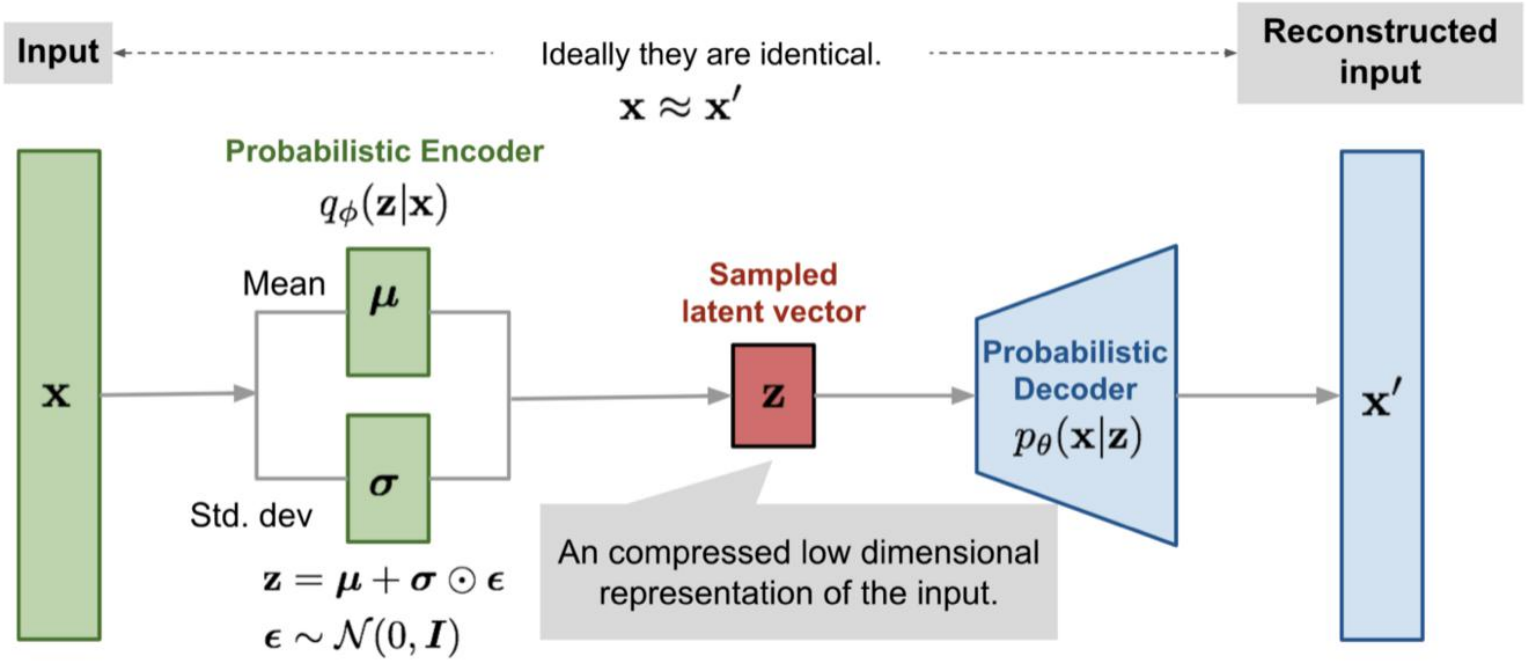
Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

VAE

1. Encoder-Decoer model





Variational Auto Encoder global architecture

VAE-Setting

Dataset: $X = \{x^{(i)}\}_{i=1}^{N}$

Aim： maximize $\log p(X) = \sum_{i=1}^{N} \log p(x^{(i)})$

Using unobserved continuous random variable z

1) $z^{(i)}$ is generated from some prior distribution $p_{\theta^*}(z)$
2) $x^{(i)}$ is generated from some conditional distribution $p_{\theta}^{*}(x \mid z)$

VAE-Setting

Problem: unknown distributions $p_\theta(z) \quad p_\theta(x \mid z)$

Solution: $q_\phi(z) \sim p_\theta(z)$

$$
\begin{aligned}
D_{KL}[q(z \mid x) \| p(z \mid x)] &= E_{z \sim q(z|x)}[\log q(z \mid x) - \log p(z \mid x)] \\
&= E_{z \sim q(z|x)}[\log q(z \mid x) - \log p(x \mid z) - \log p(z)] + \log p(x)
\end{aligned}
$$

$$p(z \mid x) = \frac{p(x \mid z) p(z)}{p(x)}$$

$$
\begin{aligned}
\log p(\mathbf{x}) &\geq \log p(\mathbf{x}) - D_{KL}\left(q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right) \\
&= E_{z \sim q(z|x)}[\log p(x \mid z) - D_{KL}(q(z \mid x) \| p(z))] = \mathcal{L}_{\theta,\phi}(\mathbf{x})
\end{aligned}
$$

Evidence Lower Bound(ELBO) or Variational Lower Bound(VLB)

VAE-Gradient

$q_\phi(\mathbf{z}|\mathbf{x})$: Inference model
$p_\theta(\mathbf{x}, \mathbf{z})$: Generative model

Generative Model:

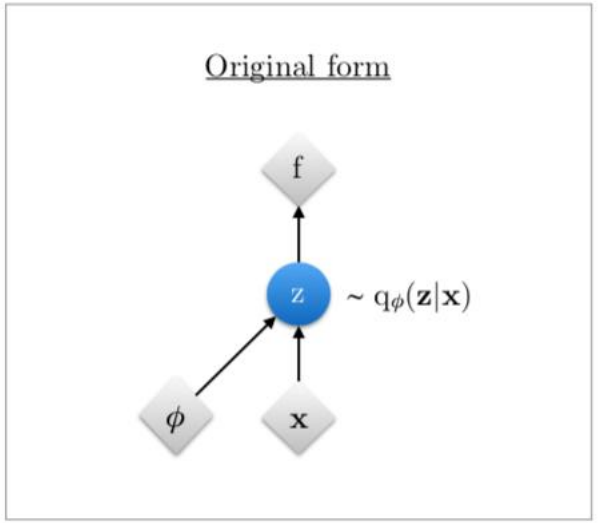
$$\begin{aligned}
\nabla_\theta \mathcal{L}_{\theta,\phi}(\mathbf{x}) &= \nabla_\theta \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z}|\mathbf{x})\right] \\
&= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\nabla_\theta(\log p_\theta(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z}|\mathbf{x}))\right] \\
&\simeq \nabla_\theta(\log p_\theta(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z}|\mathbf{x})) \\
&= \nabla_\theta(\log p_\theta(\mathbf{x}, \mathbf{z}))
\end{aligned}$$

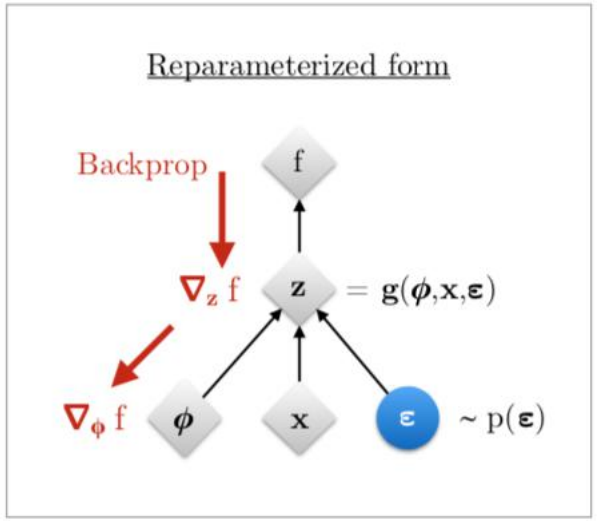
Inference Model:

$$\begin{aligned}
\nabla_\phi \mathcal{L}_{\theta,\phi}(\mathbf{x}) &= \nabla_\phi \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z}|\mathbf{x})\right] \\
&\neq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\nabla_\phi(\log p_\theta(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z}|\mathbf{x}))\right]
\end{aligned}$$

VAE-Gradient

Solution: Reparameterization Trick





$$\tilde{\mathbf{z}} = g_\phi(\boldsymbol{\epsilon}, \mathbf{x}) \quad \text{with} \quad \boldsymbol{\epsilon} \sim p(\boldsymbol{\epsilon})$$

$$\begin{aligned}\nabla_\phi \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[f(\mathbf{z})\right] &= \nabla_\phi \mathbb{E}_{p(\boldsymbol{\epsilon})}\left[f(\mathbf{z})\right] \\ &= \mathbb{E}_{p(\boldsymbol{\epsilon})}\left[\nabla_\phi f(\mathbf{z})\right] \\ &\simeq \nabla_\phi f(\mathbf{z})\end{aligned}$$

VAE-Algorithm

Algorithm 1: Stochastic optimization of the ELBO. Since noise originates from both the minibatch sampling and sampling of $p(\boldsymbol{\epsilon})$, this is a doubly stochastic optimization procedure. We also refer to this procedure as the *Auto-Encoding Variational Bayes* (AEVB) algorithm.

Data:

$\mathcal{D}$: Dataset

$q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$: Inference model

$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})$: Generative model

Result:

$\boldsymbol{\theta}, \boldsymbol{\phi}$: Learned parameters

$(\boldsymbol{\theta}, \boldsymbol{\phi}) \leftarrow$ Initialize parameters

while *SGD not converged* **do**

  $\mathcal{M} \sim \mathcal{D}$ (Random minibatch of data)

  $\boldsymbol{\epsilon} \sim p(\boldsymbol{\epsilon})$ (Random noise for every datapoint in $\mathcal{M}$)

  Compute $\tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathcal{M}, \boldsymbol{\epsilon})$ and its gradients $\nabla_{\boldsymbol{\theta},\boldsymbol{\phi}} \tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathcal{M}, \boldsymbol{\epsilon})$

  Update $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$ using SGD optimizer

end

VAE-Compute ELBO

$$E_{z\sim q(z|x)}[\log q(z \mid x) - \log p(x \mid z) - \log p(z)]$$

compute $\log q(z \mid x)$

$$\log q_{\phi}(\mathbf{z}|\mathbf{x}) = \log p(\boldsymbol{\epsilon}) - \log d_{\phi}(\mathbf{x}, \boldsymbol{\epsilon})$$

$$\log d_{\phi}(\mathbf{x}, \boldsymbol{\epsilon}) = \log \left| \det \left(\frac{\partial \mathbf{z}}{\partial \boldsymbol{\epsilon}} \right) \right|$$

当样本量N增加时，后验分布在一般条件下渐近趋于正态分布

VAE-Compute ELBO

Factorized Gaussian posteriors

$$\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$$
$$(\boldsymbol{\mu}, \log \boldsymbol{\sigma}) = \text{EncoderNeuralNet}_{\boldsymbol{\phi}}(\mathbf{x})$$
$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$$

$$\log q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) = \log p(\boldsymbol{\epsilon}) - \log d_{\boldsymbol{\phi}}(\mathbf{x}, \boldsymbol{\epsilon})$$
$$= \sum_i \log \mathcal{N}(\epsilon_i; 0, 1) - \log \sigma_i$$

Full-covariance Gaussian posterior

$$\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$$
$$\mathbf{z} = \boldsymbol{\mu} + \mathbf{L}\boldsymbol{\epsilon}$$

$$\log q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) = \log p(\boldsymbol{\epsilon}) - \sum_i \log |L_{ii}|$$

$$(\boldsymbol{\mu}, \log \boldsymbol{\sigma}, \mathbf{L}') \leftarrow \text{EncoderNeuralNet}_{\boldsymbol{\phi}}(\mathbf{x})$$
$$\mathbf{L} \leftarrow \mathbf{L}_{mask} \odot \mathbf{L}' + \text{diag}(\boldsymbol{\sigma})$$

VAE-Encoder&Decoder

C.1 Bernoulli MLP as decoder

In this case let $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})$ be a multivariate Bernoulli whose probabilities are computed from $\mathbf{z}$ with a fully-connected neural network with a single hidden layer:

$$\begin{aligned} \log p(\mathbf{x}|\mathbf{z}) &= \sum_{i=1}^{D} x_i \log y_i + (1 - x_i) \cdot \log(1 - y_i) \\ \text{where } \ \mathbf{y} &= f_\sigma(\mathbf{W}_2 \tanh(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2) \end{aligned} \tag{11}$$

where $f_\sigma(.)$ is the elementwise sigmoid activation function, and where $\boldsymbol{\theta} = \{\mathbf{W}_1, \mathbf{W}_2, \mathbf{b}_1, \mathbf{b}_2\}$ are the weights and biases of the MLP.

C.2 Gaussian MLP as encoder or decoder

In this case let encoder or decoder be a multivariate Gaussian with a diagonal covariance structure:

$$\begin{aligned} \log p(\mathbf{x}|\mathbf{z}) &= \log \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\sigma}^2 \mathbf{I}) \\ \text{where } \ \boldsymbol{\mu} &= \mathbf{W}_4 \mathbf{h} + \mathbf{b}_4 \\ \log \boldsymbol{\sigma}^2 &= \mathbf{W}_5 \mathbf{h} + \mathbf{b}_5 \\ \mathbf{h} &= \tanh(\mathbf{W}_3 \mathbf{z} + \mathbf{b}_3) \end{aligned} \tag{12}$$

where $\{\mathbf{W}_3, \mathbf{W}_4, \mathbf{W}_5, \mathbf{b}_3, \mathbf{b}_4, \mathbf{b}_5\}$ are the weights and biases of the MLP and part of $\boldsymbol{\theta}$ when used as decoder. Note that when this network is used as an encoder $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$, then $\mathbf{z}$ and $\mathbf{x}$ are swapped, and the weights and biases are variational parameters $\boldsymbol{\phi}$.

VAE-Algorithm

$$E_{z\sim q(z|x)}[\log q(z \mid x) - \log p(x \mid z) - \log p(z)]$$

Algorithm 2: Computation of unbiased estimate of single-datapoint ELBO for example VAE with a full-covariance Gaussian inference model and a factorized Bernoulli generative model. $\mathbf{L}_{mask}$ is a masking matrix with zeros on and above the diagonal, and ones below the diagonal.

Data:

$\mathbf{x}$: a datapoint, and optionally other conditioning information

$\boldsymbol{\epsilon}$: a random sample from $p(\boldsymbol{\epsilon}) = \mathcal{N}(0, \mathbf{I})$

$\boldsymbol{\theta}$: Generative model parameters

$\boldsymbol{\phi}$: Inference model parameters

$q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$: Inference model

$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})$: Generative model

Result:

$\tilde{\mathcal{L}}$: unbiased estimate of the single-datapoint ELBO $\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x})$

$(\boldsymbol{\mu}, \log \boldsymbol{\sigma}, \mathbf{L}') \leftarrow \text{EncoderNeuralNet}_{\boldsymbol{\phi}}(\mathbf{x})$

$\mathbf{L} \leftarrow \mathbf{L}_{mask} \odot \mathbf{L}' + \text{diag}(\boldsymbol{\sigma})$

$\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$

$\mathbf{z} \leftarrow \mathbf{L}\boldsymbol{\epsilon} + \boldsymbol{\mu}$

$\tilde{\mathcal{L}}_{\text{logqz}} \leftarrow -\sum_i(\frac{1}{2}(\epsilon_i^2 + \log(2\pi) + \log \sigma_i))_i$ $\quad \triangleright = q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$

$\tilde{\mathcal{L}}_{\text{logpz}} \leftarrow -\sum_i(\frac{1}{2}(z_i^2 + \log(2\pi)))$ $\quad \triangleright = p_{\boldsymbol{\theta}}(\mathbf{z})$

$\mathbf{p} \leftarrow \text{DecoderNeuralNet}_{\boldsymbol{\theta}}(\mathbf{z})$

$\tilde{\mathcal{L}}_{\text{logpx}} \leftarrow \sum_i(x_i \log p_i + (1 - x_i)\log(1 - p_i))$ $\quad \triangleright = p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})$

$\tilde{\mathcal{L}} = \tilde{\mathcal{L}}_{\text{logpx}} + \tilde{\mathcal{L}}_{\text{logpz}} - \tilde{\mathcal{L}}_{\text{logqz}}$

VAE-Results

MNIST and Frey Face datasets

Frey Face datasets:

From Brendan Frey. Almost 2000 images of Brendan's face, taken from sequential frames of a small video. Size: 20x28

Variational Lower Bound, and the estimated marginal likelihood.

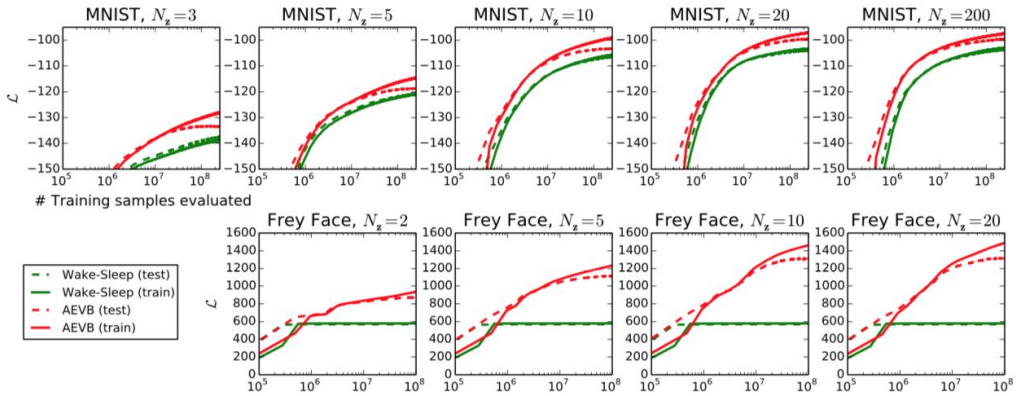


Figure 2: Comparison of our AEVB method to the wake-sleep algorithm, in terms of optimizing the lower bound, for different dimensionality of latent space ($N_{\mathbf{z}}$). Our method converged considerably faster and reached a better solution in all experiments. Interestingly enough, more latent variables does not result in more overfitting, which is explained by the regularizing effect of the lower bound. Vertical axis: the estimated average variational lower bound per datapoint. The estimator variance was small (< 1) and omitted. Horizontal axis: amount of training points evaluated. Computation took around 20-40 minutes per million training samples with a Intel Xeon CPU running at an effective 40 GFLOPS.

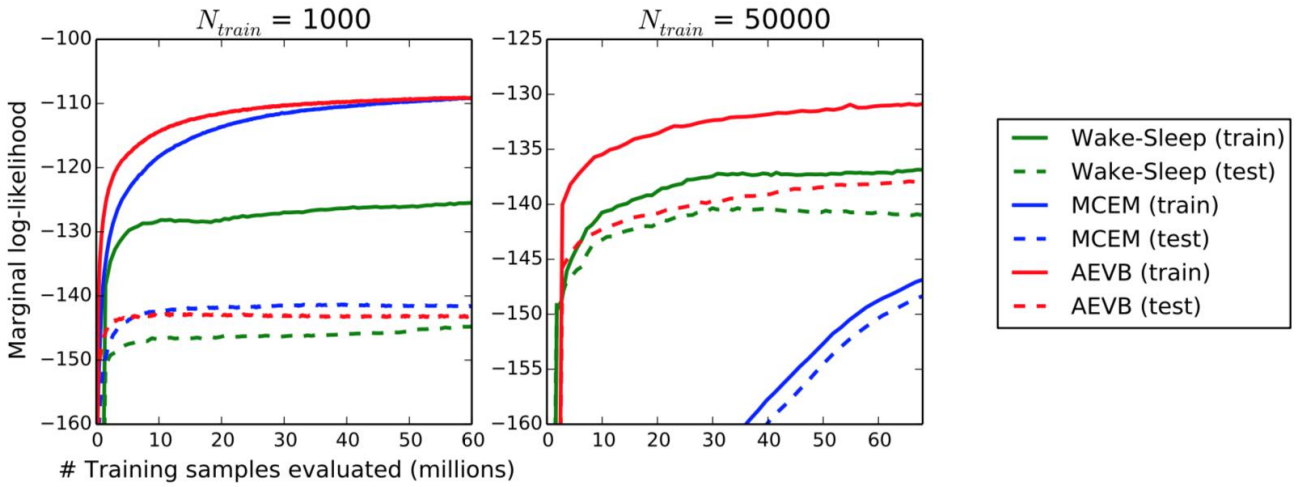
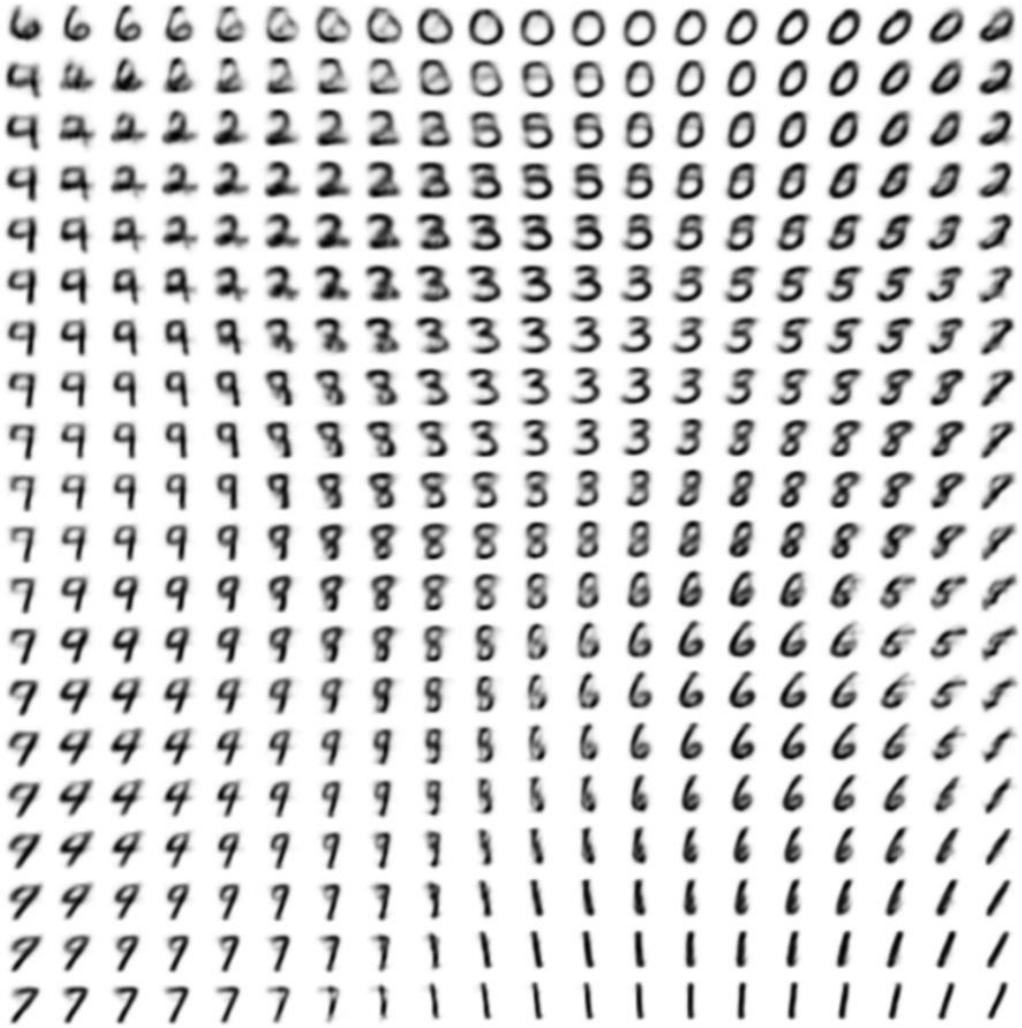


Figure 3: Comparison of AEVB to the wake-sleep algorithm and Monte Carlo EM, in terms of the estimated marginal likelihood, for a different number of training points. Monte Carlo EM is not an on-line algorithm, and (unlike AEVB and the wake-sleep method) can't be applied efficiently for the full MNIST dataset.

VAE-Results

(a) Learned Frey Face manifold



(b) Learned MNIST manifold

IAF

Improved Variational Inference with Inverse Autoregressive Flow

Diederik P. Kingma
dpkingma@openai.com

Tim Salimans
tim@openai.com

Rafal Jozefowicz
rafal@openai.com

Xi Chen
peter@openai.com

Ilya Sutskever
ilya@openai.com

Max Welling*
M.Welling@uva.nl

IAF

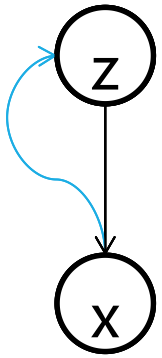
Aim: improving the flexibility of the inference model

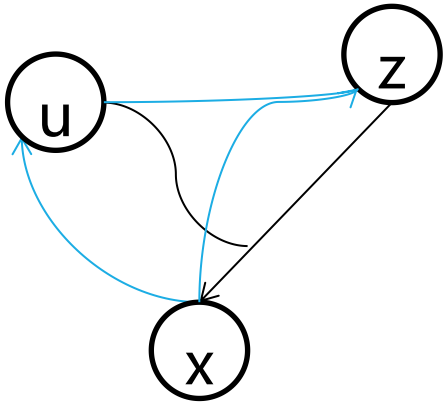
(1) efficient to compute and differentiate its probability density $q\varphi(z|x)$

(2) efficient to sample from

(3) parallel computational

Usually: Gaussian posteriors





Solution: 1) **Auxiliary Latent Variables**

2) Normalizing Flows

IAF-Normalizing Flows

2) Normalizing Flows

$$\log p(\mathbf{x}) \geq \log p(\mathbf{x}) - D_{KL}\left(q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})\right)$$
$$= E_{z \sim q(z|x)}[\log p(x \mid z) - D_{KL}(q(z \mid x) \| p(z))] = \mathcal{L}_{\theta,\phi}(\mathbf{x})$$

$$\boldsymbol{\epsilon}_0 \sim p(\boldsymbol{\epsilon})$$
$$\text{for } t = 1 \ldots T:$$
$$\boldsymbol{\epsilon}_t = \mathbf{f}_t(\boldsymbol{\epsilon}_{t-1}, \mathbf{x})$$
$$\mathbf{z} = \boldsymbol{\epsilon}_T$$

$$\frac{d\mathbf{z}}{d\boldsymbol{\epsilon}_0} = \prod_{t=1}^{T} \frac{d\boldsymbol{\epsilon}_t}{d\boldsymbol{\epsilon}_{t-1}}$$

$$\log q_\phi(\mathbf{z}|\mathbf{x}) = \log p(\boldsymbol{\epsilon}_0) - \sum_{t=1}^{T} \log \det \left| \frac{d\boldsymbol{\epsilon}_t}{d\boldsymbol{\epsilon}_{t-1}} \right|$$

$$\mathbf{f}_t(\mathbf{z}_{t-1}) = \mathbf{z}_{t-1} + \mathbf{u}h(\mathbf{w}^T \mathbf{z}_{t-1} + b)$$

IAF

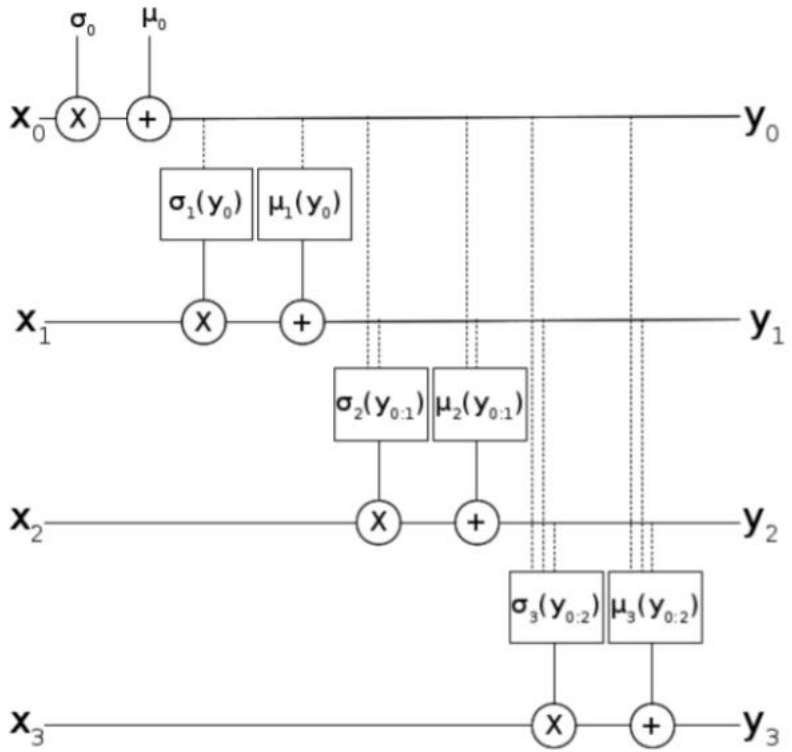
Problem: Not scale well to a high-dimensional latent space:

Inspiration: autoregressive autoencoders such as MADE and the PixelCNN

$$\mathbf{y} = \{y_i\}_{i=1}^{D} \qquad [\boldsymbol{\mu}(\mathbf{y}), \boldsymbol{\sigma}(\mathbf{y})]$$

$$\partial[\boldsymbol{\mu}_i, \boldsymbol{\sigma}_i]/\partial \mathbf{y}_j = [0, 0] \quad \text{for } j \geq i.$$

$$y_0 = \mu_0 + \sigma_0 \odot \epsilon_0, \text{ and for } i > 0,\ y_i = \mu_i(\mathbf{y}_{1:i-1}) + \sigma_i(\mathbf{y}_{1:i-1}) \cdot \epsilon_i.$$



Autoregressive Transform

IAF-Inverse

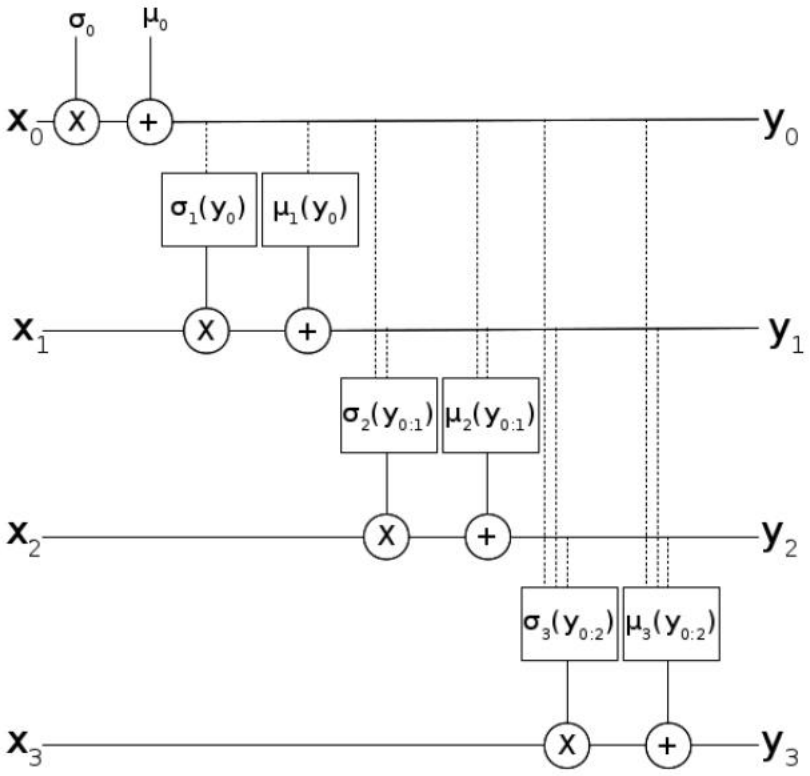
$$\epsilon_i = \frac{y_i - \mu_i(\mathbf{y}_{1:i-1})}{\sigma_i(\mathbf{y}_{1:i-1})}$$

$$\boldsymbol{\epsilon} = (\mathbf{y} - \boldsymbol{\mu}(\mathbf{y}))/\boldsymbol{\sigma}(\mathbf{y})$$

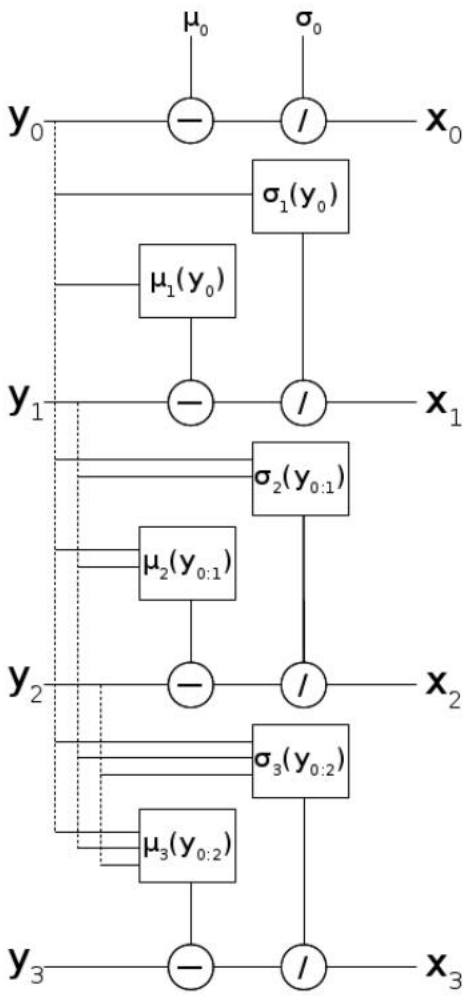
$$\log\det\left|\frac{d\boldsymbol{\epsilon}}{d\mathbf{y}}\right| = \sum_{i=1}^{D} -\log\sigma_i(\mathbf{y})$$

$$\boldsymbol{\epsilon} = \boldsymbol{\sigma}(\mathbf{y}) \odot \mathbf{y} + \boldsymbol{\mu}(\mathbf{y})$$

$$\log\det\left|\frac{d\boldsymbol{\epsilon}}{d\mathbf{y}}\right| = \sum_{i=1}^{D} \log\sigma_i(\mathbf{y})$$



Autoregressive Transform



Inverse Autoregressive Transform

IAF-Algorithm

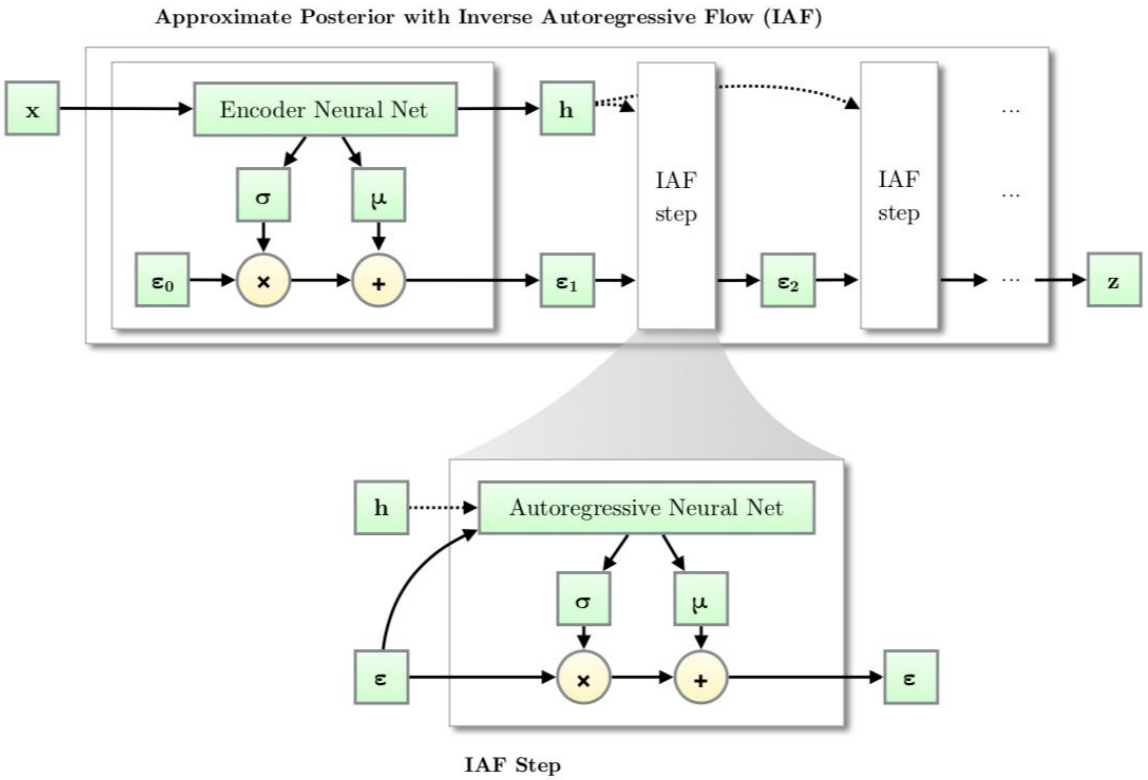


Figure 3.1: Like other normalizing flows, drawing samples from an approximate posterior with Inverse Autoregressive Flow (IAF) (Kingma *et al.*, 2016) starts with a distribution with tractable density, such as a Gaussian with diagonal covariance, followed by a chain of nonlinear invertible transformations of $\mathbf{z}$, each with a simple Jacobian determinant. The final iterate has a flexible distribution.

$$\boldsymbol{\epsilon}_0 \sim \mathcal{N}(0, I)$$

$$(\boldsymbol{\mu}_0, \log \boldsymbol{\sigma}_0, \mathbf{h}) = \text{EncoderNeuralNet}(\mathbf{x}; \boldsymbol{\theta})$$

$$\mathbf{z}_0 = \boldsymbol{\mu}_0 + \boldsymbol{\sigma}_0 \odot \boldsymbol{\epsilon}_0$$

$$(\boldsymbol{\mu}_t, \boldsymbol{\sigma}_t) = \text{AutoregressiveNeuralNet}_t(\boldsymbol{\epsilon}_{t-1}, \mathbf{h}; \boldsymbol{\theta})$$

$$\boldsymbol{\epsilon}_t = \boldsymbol{\mu}_t + \boldsymbol{\sigma}_t \odot \boldsymbol{\epsilon}_{t-1}$$

$$\mathbf{z} \equiv \boldsymbol{\epsilon}_T$$

$$\log q(\mathbf{z}|\mathbf{x}) = -\sum_{i=1}^{D} \left(\tfrac{1}{2}\epsilon_i^2 + \tfrac{1}{2}\log(2\pi) + \sum_{t=0}^{T} \log \sigma_{t,i} \right)$$

$$(\mathbf{m}_t, \mathbf{s}_t) = \text{AutoregressiveNeuralNet}_t(\boldsymbol{\epsilon}_{t-1}, \mathbf{h}; \boldsymbol{\theta})$$

$$\boldsymbol{\sigma}_t = \text{sigmoid}(\mathbf{s}_t)$$

$$\boldsymbol{\epsilon}_t = \boldsymbol{\sigma}_t \odot \boldsymbol{\epsilon}_{t-1} + (1 - \boldsymbol{\sigma}_t) \odot \mathbf{m}_t$$

IAF-Algorithm

Algorithm 3: Pseudo-code of an approximate posterior with Inverse Autoregressive Flow (IAF).

Input: $\texttt{EncoderNN}(\mathbf{x};\boldsymbol{\theta})$ is an encoder neural network, with additional output $\mathbf{h}$.

Input: $\texttt{AutoregressiveNN}(\mathbf{z};\mathbf{h},t,\boldsymbol{\theta})$ is a neural network that is autoregressive over $\mathbf{z}$, with additional inputs $\mathbf{h}$ and t.

Input: T signifies the number of steps of flow.

Data:

$\mathbf{x}$: a datapoint, and optionally other conditioning information

$\boldsymbol{\theta}$: neural network parameters

Result:

$\mathbf{z}$: a random sample from $q(\mathbf{z}|\mathbf{x})$, the approximate posterior distribution

l: the scalar value of $\log q(\mathbf{z}|\mathbf{x})$, evaluated at sample '$\mathbf{z}$'

$[\boldsymbol{\mu},\boldsymbol{\sigma},\mathbf{h}] \leftarrow \texttt{EncoderNN}(\mathbf{x};\boldsymbol{\theta})$

$\boldsymbol{\epsilon} \sim \mathcal{N}(0,I)$

$\mathbf{z} \leftarrow \boldsymbol{\sigma} \odot \boldsymbol{\epsilon} + \boldsymbol{\mu}$

$l \leftarrow -\sum_i(\log\sigma_i + \frac{1}{2}\epsilon_i^2 + \frac{1}{2}\log(2\pi))$

for $t \leftarrow 1$ **to** T **do**

$\quad[\mathbf{m},\mathbf{s}] \leftarrow \texttt{AutoregressiveNN}(\mathbf{z};\mathbf{h},t,\boldsymbol{\theta})$

$\quad\boldsymbol{\sigma} \leftarrow (1+\exp(-\mathbf{s}))^{-1}$

$\quad\mathbf{z} \leftarrow \boldsymbol{\sigma} \odot \mathbf{z} + (1-\boldsymbol{\sigma}) \odot \mathbf{m}$

$\quad l \leftarrow l - \sum_i(\log\sigma_i)$

end

IAF-Result

Table 1: Generative modeling results on the dynamically sampled binarized MNIST version used in previous publications (Burda et al., 2015). Shown are averages; the number between brackets are standard deviations across 5 optimization runs. The right column shows an importance sampled estimate of the marginal likelihood for each model with 128 samples. Best previous results are reproduced in the first segment: [1]: (Salimans et al., 2014) [2]: (Burda et al., 2015) [3]: (Kaae Sønderby et al., 2016) [4]: (Tran et al., 2015)

Model	VLB	$\log p(\mathbf{x}) \approx$
Convolutional VAE + HVI [1]	-83.49	-81.94
DLGM 2hl + IWAE [2]		-82.90
LVAE [3]		-81.74
DRAW + VGP [4]	-79.88	
Diagonal covariance	-84.08 ($\pm$ 0.10)	-81.08 ($\pm$ 0.08)
IAF (Depth = 2, Width = 320)	-82.02 ($\pm$ 0.08)	-79.77 ($\pm$ 0.06)
IAF (Depth = 2, Width = 1920)	-81.17 ($\pm$ 0.08)	-79.30 ($\pm$ 0.08)
IAF (Depth = 4, Width = 1920)	-80.93 ($\pm$ 0.09)	-79.17 ($\pm$ 0.08)
IAF (Depth = 8, Width = 1920)	-80.80 ($\pm$ 0.07)	**-79.10** ($\pm$ 0.07)

Table 2: Our results with ResNet VAEs on CIFAR-10 images, compared to earlier results, in *average number of bits per data dimension* on the test set. The number for convolutional DRAW is an upper bound, while the ResNet VAE log-likelihood was estimated using importance sampling.

Method	bits/dim $\leq$
Results with tractable likelihood models:	
Uniform distribution (van den Oord et al., 2016b)	8.00
Multivariate Gaussian (van den Oord et al., 2016b)	4.70
NICE (Dinh et al., 2014)	4.48
Deep GMMs (van den Oord and Schrauwen, 2014)	4.00
Real NVP (Dinh et al., 2016)	3.49
PixelRNN (van den Oord et al., 2016b)	**3.00**
Gated PixelCNN (van den Oord et al., 2016c)	**3.03**
Results with variationally trained latent-variable models:	
Deep Diffusion (Sohl-Dickstein et al., 2015)	5.40
Convolutional DRAW (Gregor et al., 2016)	3.58
ResNet VAE with IAF (Ours)	**3.11**

$$
\begin{aligned}
b(\mathbf{x}) &= -\frac{\log_2 p_z(\mathbf{z})}{D} \\
&= -\frac{\log p(\mathbf{x})}{D \log 2} - \log_2(1-2\lambda) + 8 + \frac{1}{D}\sum_i (\log_2 \sigma(x_i) + \log_2(1-\sigma(x_i))).
\end{aligned}
$$

iVAE

Variational Autoencoders and Nonlinear ICA: A Unifying Framework

Ilyes Khemakhem
Gatsby Unit
UCL

Diederik P. Kingma
Google Brain

Ricardo Pio Monti
Gatsby Unit
UCL

Aapo Hyvärinen
Université Paris-Saclay,
Inria, Univ. of Helsinki

iVAE

Problem: marginal distribution over x is meaningful (Eq. 3). The rest of the learned distributions are, generally, quite meaningless.

Inspiration: Recent work in nonlinear Independent Component Analysis (ICA) theory

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) \mathrm{d}\mathbf{z}$$

$$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) = p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z}) p_{\boldsymbol{\theta}}(\mathbf{z})$$

$$p_{\boldsymbol{\theta}}(\mathbf{x}) \approx p_{\boldsymbol{\theta}^*}(\mathbf{x})$$

Looking for is models for which the following implication holds for all (x, z):

$$\forall(\boldsymbol{\theta}, \boldsymbol{\theta}') : \quad p_{\boldsymbol{\theta}}(\mathbf{x}) = p_{\boldsymbol{\theta}'}(\mathbf{x}) \quad \Longrightarrow \quad \boldsymbol{\theta} = \boldsymbol{\theta}'$$

$$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) = p_{\boldsymbol{\theta}^*}(\mathbf{x}, \mathbf{z}) \qquad p_{\boldsymbol{\theta}}(\mathbf{z}) = p_{\boldsymbol{\theta}^*}(\mathbf{z}) \qquad p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}) = p_{\boldsymbol{\theta}^*}(\mathbf{z}|\mathbf{x}) \qquad q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$$

iVAE

If not we can always find transformations of z that changes its value but does not change its distribution

Example: For a spherical Gaussian distribution applying a rotation keeps its distribution the same.

The primary assumption leading to identifiability is a conditionally factorized prior distribution over the latent variables $p_\theta(z|u)$, where u is an additionally observed variable

eg: index in a time series, previous data point in a time series, some kind of (possibly noisy) class label, or another concurrently observed variable.

iVAE

Formally, let $\mathbf{x} \in \mathbb{R}^d$, and $\mathbf{u} \in \mathbb{R}^m$ be two observed random variables, and $\mathbf{z} \in \mathbb{R}^n$ (lower-dimensional, $n \leq d$) a latent variable. Let $\boldsymbol{\theta} = (\mathbf{f}, \mathbf{T}, \boldsymbol{\lambda})$ be the parameters of the following conditional generative model:

$$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}|\mathbf{u}) = p_{\mathbf{f}}(\mathbf{x}|\mathbf{z}) p_{\mathbf{T},\boldsymbol{\lambda}}(\mathbf{z}|\mathbf{u}) \tag{5}$$

where we first define:

$$p_{\mathbf{f}}(\mathbf{x}|\mathbf{z}) = p_{\boldsymbol{\varepsilon}}(\mathbf{x} - \mathbf{f}(\mathbf{z})) \tag{6}$$

$$p_{\mathbf{T},\boldsymbol{\lambda}}(\mathbf{z}|\mathbf{u}) = \prod_i \frac{Q_i(z_i)}{Z_i(\mathbf{u})} \exp\left[\sum_{j=1}^{k} T_{i,j}(z_i)\lambda_{i,j}(\mathbf{u})\right]$$

$$\mathbb{E}_{q_{\mathcal{D}}}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{u})\right] \geq \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}) :=$$
$$\mathbb{E}_{q_{\mathcal{D}}}\left[\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x},\mathbf{u})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}|\mathbf{u}) - \log q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}, \mathbf{u})\right]\right] \tag{8}$$

iVAE

$\boldsymbol{\theta} = (\mathbf{f}, \mathbf{T}, \boldsymbol{\lambda})$

$p_{\mathbf{f}}(\mathbf{x}|\mathbf{z}) = p_{\boldsymbol{\varepsilon}}(\mathbf{x} - \mathbf{f}(\mathbf{z}))$

$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}|\mathbf{u}) = p_{\mathbf{f}}(\mathbf{x}|\mathbf{z}) p_{\mathbf{T},\boldsymbol{\lambda}}(\mathbf{z}|\mathbf{u})$

Notations Let $\mathcal{Z} \subset \mathbb{R}^n$ and $\mathcal{X} \subset \mathbb{R}^d$ be the domain and the image of $\mathbf{f}$ in (6), respectively, and $\mathcal{U} \subset \mathbb{R}^m$ the support of the distribution of $\mathbf{u}$. We denote by $\mathbf{f}^{-1}$ the inverse defined from $\mathcal{X} \to \mathcal{Z}$. We suppose that $\mathcal{Z}$, $\mathcal{X}$ and $\mathcal{U}$ are open sets. We denote by $\mathbf{T}(\mathbf{z}) := (\mathbf{T}_1(z_1), \ldots, \mathbf{T}_n(z_n)) = (T_{1,1}(z_1) \ldots, T_{n,k}(z_n)) \in \mathbb{R}^{nk}$ the vector of sufficient statistics of (7), $\boldsymbol{\lambda}(\mathbf{u}) = (\boldsymbol{\lambda}_1(\mathbf{u}), \ldots, \boldsymbol{\lambda}_n(\mathbf{u})) = (\lambda_{1,1}(\mathbf{u}), \ldots, \lambda_{n,k}(\mathbf{u})) \in \mathbb{R}^{nk}$ the vector of its parameters. Finally $\Theta = \{\boldsymbol{\theta} := (\mathbf{f}, \mathbf{T}, \boldsymbol{\lambda})\}$ is the domain of parameters describing (5).

Definition 1 *Let $\sim$ be an equivalence relation on Θ. We say that (1) is identifiable up to $\sim$ (or $\sim$-identifiable) if*

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = p_{\tilde{\boldsymbol{\theta}}}(\mathbf{x}) \implies \tilde{\boldsymbol{\theta}} \sim \boldsymbol{\theta} \tag{12}$$

The elements of the quotient space $\Theta/_{\sim}$ are called the identifiability classes.

Definition 2 *Let $\sim$ be the equivalence relation on Θ defined as follows:*

$$(\mathbf{f}, \mathbf{T}, \boldsymbol{\lambda}) \sim (\tilde{\mathbf{f}}, \tilde{\mathbf{T}}, \tilde{\boldsymbol{\lambda}}) \Leftrightarrow$$
$$\exists A, \mathbf{c} \mid \mathbf{T}(\mathbf{f}^{-1}(\mathbf{x})) = A\tilde{\mathbf{T}}(\tilde{\mathbf{f}}^{-1}(\mathbf{x})) + \mathbf{c}, \forall \mathbf{x} \in \mathcal{X} \tag{13}$$

where A is an $nk \times nk$ matrix and $\mathbf{c}$ is a vector

If A is invertible, *we denote this relation by $\sim_A$. If A is a block* permutation[2] *matrix, we denote it by $\sim_P$.*

iVAE

Theorem 1 *Assume that we observe data sampled from a generative model defined according to (5)-(7), with parameters* $(\mathbf{f}, \mathbf{T}, \boldsymbol{\lambda})$. *Assume the following holds:*

(i) The set $\{\mathbf{x} \in \mathcal{X} | \varphi_\varepsilon(\mathbf{x}) = 0\}$ *has measure zero, where* φ_ε *is the characteristic function of the density* p_ε *defined in (6).*

(ii) The mixing function $\mathbf{f}$ *in (6) is injective.*

(iii) The sufficient statistics $T_{i,j}$ *in (7) are differentiable almost everywhere, and* $(T_{i,j})_{1 \leq j \leq k}$ *are linearly independent on any subset of* $\mathcal{X}$ *of measure greater than zero.*

(iv) There exist $nk+1$ *distinct points* $\mathbf{u}^0, \ldots, \mathbf{u}^{nk}$ *such that the matrix*

$$L = (\boldsymbol{\lambda}(\mathbf{u}_1) - \boldsymbol{\lambda}(\mathbf{u}_0), \ldots, \boldsymbol{\lambda}(\mathbf{u}_{nk}) - \boldsymbol{\lambda}(\mathbf{u}_0)) \quad (14)$$

of size $nk \times nk$ *is invertible.*[4]

then the parameters $(\mathbf{f}, \mathbf{T}, \boldsymbol{\lambda})$ *are* $\sim_A$*-identifiable.*